

Research Internship:
Incorporating runtime monitors generated by ComMA into
ROS applications

Martijn Vogelaar (S1047391)

TNO Supervisor: prof. dr. J.J.M. Hooman
University Supervisor: dr. M.T.W. Schuts
Second reader: dr. E. Poll

July 12, 2022

Contents

1	Introduction	4
2	Introduction to ROS	5
2.1	ROS communication patterns	5
2.1.1	Topics	5
2.1.2	Services	5
2.1.3	Actions	6
3	Case study	7
3.1	EnemyDetector	8
3.2	LearningNode	8
3.3	StanleyController	8
3.4	control_convert node	8
3.5	telemetry_converter node	8
4	Related work	9
4.1	ROSMonitoring	9
4.2	ROSRV	10
5	Capturing data sent between ROS nodes	11
5.1	Instrumenting ROS nodes	11
5.2	ROS APIs	12
5.3	Eavesdropping messages	12
5.4	Intercepting all messages	13
5.5	Conclusion	13
6	Capturing message - implementation	14
6.1	Deserializing the captured data	15
6.1.1	rosidl_runtime_py	15
6.1.2	rcipy	15
7	Generating ComMA traces from ROS data	16
7.1	Primitive datatypes	16
7.2	Nested messages	16
7.3	Translating Topics	17
7.4	Translating Services	17
7.5	Translating Actions	18
7.6	Generating the traces	18
8	Online monitor feedback	20
8.1	What should happen when the monitor finds non-conformance?	20
8.2	Performance of the online monitor	20
8.3	What should the feedback look like?	21
9	ComMA model for the case study	23
9.1	Component: Traxxas	23
9.2	Interface: ITraxxasControl	24
9.3	Interface: ILocation	24
9.4	Interface: IWaypoint	26

10 Known issues and Limitations	27
10.1 Order of messages	27
10.2 Limited to Eprosima DDS	27
11 Evaluation	28
11.1 Activities	28
11.1.1 Familiarizing myself with ComMA and ROS	28
11.1.2 Answering the first two sub-questions	28
11.1.3 Implementing the ComMA model for the case study.	29
11.2 Results	29
11.2.1 Improving ComMA's online monitor	29
11.2.2 Incorrect calculation of time constraints	30
11.2.3 Improvement of the online monitor	30
11.3 Reflection & Evaluation	31
11.3.1 Meetings	31
11.3.2 Making notations	31
11.3.3 Overall thoughts	31
Appendices	32
A ROS Nodes architecture of the case study	32
B TShark arguments used to capture ROS packets	33
C Output of a sniffed packet by TShark	34
D TShark arguments used to capture ROS packets	35
E Configuration file for ROS2ComMA	36
F Records before	37
G Records after	38

1 Introduction

ROS [1], Robot Operating System is, unlike the name would suggest not an operating system but rather a collection of software frameworks for robotic software development. ROS allows one to create stand-alone nodes for certain parts of a system. These nodes can easily communicate with one another using the standardized ROS communication protocols. ROS nodes can be deployed on the same system but can also be deployed across various systems.

A system that uses ROS often consists of many nodes, each node has a specific task that it is supposed to do. Testing these nodes individually, can, for example, be done using unit tests.

It already becomes more difficult if we want to test the nodes while they interact with one another. Even if we do manage to thoroughly test the entire system consisting of many ROS nodes we cannot guarantee that the system is flawless.

This is where ComMA [2] comes in, ComMA allows one to specify the specification of interfaces of a software component, which in our case will be ROS nodes. ComMA is able to check whether the implemented interfaces are in conformance with these specifications. One can, among other things specify the following [2]:

- a signature, i.e. the set of commands, signals and notifications that are offered by a server or client;
- a protocol state machine that describes the allowed sequences of interaction events between clients and server. Events are occurrences of commands, signals and notifications, these events can optionally contain values;
- timing constraints on the occurrence of the events and
- data constraints on the values communicated.

ComMA is at the moment, mostly used for offline monitoring, i.e. check whether the system behaved correctly. This check is done after the system was used. An online monitor [3] e.g. runtime monitor is, however, being developed. An online monitor would allow one to monitor the system while it is running, instead of checking if the system behaved correctly after it has finished running.

The aim of the research internship and this report is to explore the usage of ComMA's online monitoring with systems developed using ROS. Part of this research will include the actual implementation of a program that allows one to monitor a system that is using ROS. To test this, a case study will be used. In this case study, a UGV (Unmanned Guided Vehicle) will drive autonomously in a simulation. A more detailed description of the case study will be given later on. The following research question has been formulated:

RQ-1 How can ComMA's runtime monitoring be applied to ROS?

To support this research question the following sub-questions have been formulated:

- RQ 1.1 - Has any related work about runtime monitoring for ROS been carried out?
- RQ 1.2 - How can data sent between ROS nodes be captured?
- RQ 1.3 - How can ComMA-formatted traces be generated from the data of ROS nodes?

2 Introduction to ROS

As previously mentioned in the introduction, a system using ROS often consists of many different ROS nodes. Nodes are often responsible for a very small part of the system, for example, publishing the values of a specific sensor or controlling a stepper-motor. Nodes can easily be used in different systems as the nodes are relatively small and only have a single task. Nodes can communicate with one another using Topics, Services and/or Actions which will be explained later. There are currently two versions of ROS, ROS 1 and ROS 2. While the main philosophy behind ROS remained the same, ROS 2 is for a large part an improvement of ROS 1 as it began to show its limitations[4]. In this report, any reference to ROS will be a reference to ROS 2.

2.1 ROS communication patterns

As mentioned before, communication between different ROS nodes can be achieved with multiple patterns. A brief description of the different types will be given below.

2.1.1 Topics

ROS topics [5] are equal to the well-known Publisher-Subscriber pattern. Node *A* publishes messages on topic *Z*. Any node that knows that topic *Z* exists can subscribe to this topic and receive the messages published by *A*.

Topics are defined in files with the “.msg” extension and defined the following format:

```
int32 sensor1_value
int32 sensor2_value
int32 sensor3_value
```

A message on a topic can contain multiple fields as can be seen above, each new field has to be defined on a new line.

2.1.2 Services

ROS services [6] are equal to the well-known Request-Reply pattern. Node *A* and Node *B* exist, *B* offers service *Z*. When *A* wants service *Z* it can send a message to *B* to request the service, *B* will then send a response to *A*.

Services are defined in files with the “.srv” extension and defined in the following format:

```
int32 request_value1
int32 request_value2
---
int32 response_value1
int32 response_value2
```

The three dashes separate the request and response as both the request and response are defined in the same file. Similar to the messages, multiple fields can be added to a request or result by adding a new line with datatype and field name.

2.1.3 Actions

ROS actions [7] are very similar to ROS services, the only difference is that in between the request and the response, updates can be sent.

Actions are defined in files with the ".act" extension and used in the following format:

```
int32 request
---
int32 response
---
int32 update
```

Similar to the ROS service, the three dashes separate the request, response and update parts of the action as all three are defined within the same file. An example flow of the usage of an action can be seen below.

Node B offers action Z	
Node A requests action Z from Node B	request: 10
Node B updates Node A	update: 6
Node B updates Node A	update: 7
Node B updates Node A	update: 8
Node B updates Node A	update: 9
Node B updates Node A	update: 10
Node B is finished and sends a response to Node A.	response: 10

3 Case study

A case study will be used to have a system that can be monitored by ComMA. The case study that will be used contains a Traxxas, a radio controlled car, that acts as a UGV (Unmanned Guided Vehicle) that surveils a demarcated area. The UGV surveils this area as enemies can appear. Initially, the UGV has no smart strategy to surveil the area. The UGV gradually gets to know the environment and locates spots where enemies are more likely to appear. Besides this, the UGV is also able to locate obstacles(water and sand) that impose a penalty if travelled through. During the surveillance of the area, the UGV learns to drive more efficient routes with a higher chance of finding enemies.

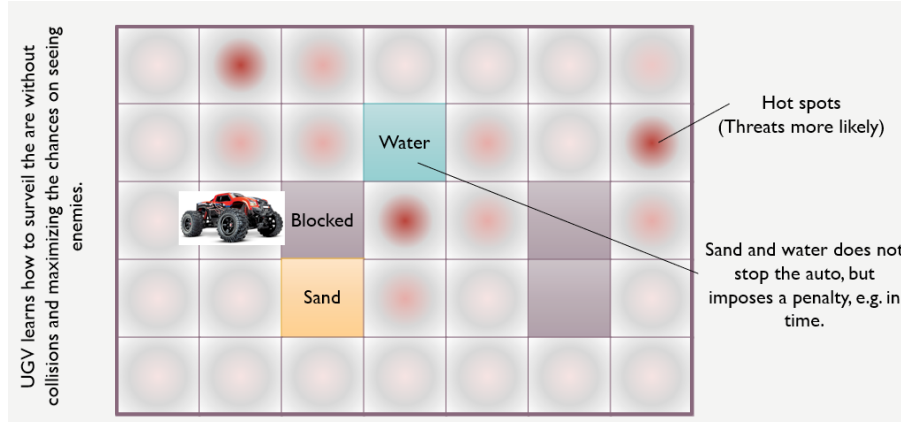


Figure 1: Case study visualization

The case study is implemented with ROS as the underlying framework, Unity 3D is used to simulate the setup. The simulation made in Unity 3D can be seen in Figure 2. Besides the simulation, there is also a GUI that shows the current location of the UGV in green and the route that has to be followed to the next waypoint in purple. The GUI can furthermore be used to manually send the UGV to a specific location by clicking on the grid.

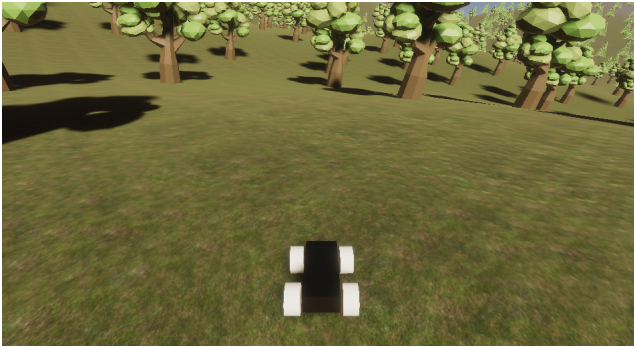


Figure 2: Case study simulation

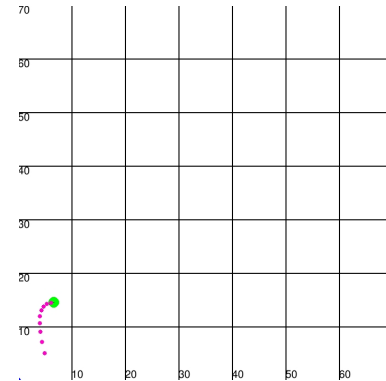


Figure 3: Case study GUI

In appendix A an updated version of the original[8] ROS architecture of the case study can be found. The original ROS architecture did not include all ROS nodes currently present in the simulation. All red rectangles represent ROS nodes, the most important ROS nodes will be described below.

3.1 EnemyDetector

The EnemyDetector is a ROS node located in the Unity Simulation. It receives a next_waypoint message from the LearningNode by listening to the next_waypoint topic. If the Traxxas gets close enough to the waypoint, a message will be published on the enemy_detector topic.

3.2 LearningNode

The LearningNode is the ROS node that controls the direction that will be driven towards. Depending on the observations made by the EnemyDetector and the current location, a new waypoint is calculated. This waypoint is then published on the next_waypoint topic that the EnemyDetector and the StanleyController are listening to.

3.3 StanleyController

The StanleyController is a ROS node that is able to calculate the steering angle and throttle necessary to drive towards the waypoint. This calculation is based on the current location, speed and attitude (roll, pitch, yaw) i.e. orientation in space, of the UGV. The necessary steering angle and throttle are then sent towards the control_converter node via ROS services.

3.4 control_convert node

The control_convert node converts the steering angle and throttle messages into a single message that can be used by the Unity 3D Simulation.

3.5 telemetry_converter node

The telemetry_convert node converts the Unity 3D telemetry message into a location, attitude and velocity message. These are then sent over a ROS topic that other nodes are listening to.

4 Related work

One of the problems currently faced is RQ-1.3: How can data sent between ROS nodes be captured? To be able to monitor the interfaces and behavior of ROS nodes at runtime the communication between the nodes has to be caught. Related work might help solve this problem as these could have faced this same or a similar problem. In this chapter different related work will be discussed and analyzed.

4.1 ROSMonitoring

ROSMonitoring [9] is a framework developed to be able to, at runtime, verify messages exchanged within a system consisting of ROS nodes. Besides the online monitoring (runtime monitoring) ROSMonitoring also supports offline monitoring which allows verifying the system after it has run. ROSMonitoring has been created for ROS 1, even though it is an older version, the general patterns of ROS stayed the same between ROS 1 and ROS 2 which is why it is still useful to look into ROSMonitoring. This does however not mean that ROSMonitoring can be used for ROS 2.

To be able to capture mutual communication between ROS nodes, ROSMonitoring uses instrumentation. One is able to configure which nodes and which message will be monitored using yaml-files. Using the configuration files, ROSMonitoring instruments the specified nodes to send each specified message twice, once to the actual destination and once to a monitoring node. This monitor node, which is also a ROS node, converts the ROS message to a message conform RML(Runtime Monitoring Language) [10].

If the monitor is used as online monitor it sends the messages to a RMLOracle that implements the Runtime Monitoring Language. If non-conformity is found, feedback will be published via a ROS topic. Otherwise, it will output the messages to a log file which can be easily analyzed later on.

Due to the fact that ROSMonitoring uses instrumentation of ROS nodes to capture messages, it currently only works for ROS nodes implemented in Python. Nodes implemented in other languages can thus not be monitored. Each language will require its own implementation of the instrumentation. In the case study most ROS nodes are written in Python. Nodes that are used to control the simulation are however written in C.

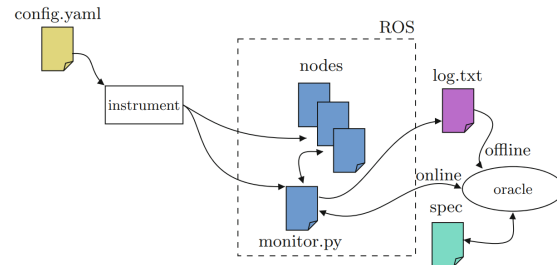


Fig. 1. High-level overview of ROSMonitoring.

Figure 4: High-level overview of ROSMonitoring (Ferrando et al. [11])

4.2 ROSRV

ROSRV is a runtime verification framework for robotic applications on top of the Robot Operating System [12]. ROSRV, unlike ROSMonitor, does not only monitor messages, it is also able to alter messages if a safety property is not met. An example given by Huang et al. [12] is the following: to prevent a robot from overturn, a safety monitor can intercept and modify the robot speed/acceleration messages.

ROSRV is thus no longer only a runtime verification framework but it is also able to intervene based on safety properties specified. Besides this, ROSRV is also able to enforce security policies. It is for example possible to specify that only messages originating from a certain IP address can post on certain topics.

It is apparent that ROSRV is not only able to monitor the behavior like ROSMonitoring does, but is able to do a lot more. ROSRV is able to do so by implementing a monitoring layer where all messages go through. All messages are first sent to the Monitor, the monitor verifies the message and then sends it to the actual destination.

ROSRV is able to do this because ROS nodes no longer communicate with the ROS Master but with the RVMaster as can be seen in Figure 7. In ROS 1, the ROS Master provides naming and registration services to the rest of the nodes in the ROS system. It tracks publishers and subscribers to topics as well as services. The role of the Master is to enable individual ROS nodes to locate one another. Once these nodes have located each other they communicate with each other peer-to-peer. As the ROS Nodes talk to the RVMaster, the RVMaster is able to redirect every message through the monitor. The monitor then acts as a man-in-the-middle without the ROS nodes being aware. ROS 2 no longer makes use of ROS Master which means that ROSRV does not work for ROS 2.

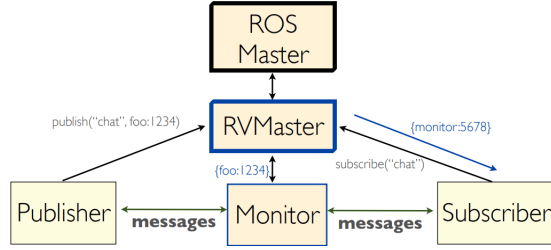


Figure 5: ROSRV Architecture (Huang et al. [12])

5 Capturing data sent between ROS nodes

To be able to use ComMA for the runtime monitoring of ROS applications, communication between the ROS nodes should somehow be sent to ComMA. The first step in this process is capturing the data being sent between the ROS nodes. In the previous section, we looked into existing solutions for runtime monitoring/verification of systems using ROS. Analyzing these existing solutions showed two possible solutions to capture data sent between ROS nodes: instrumenting each node and replacing the entire back-end of ROS.

There is currently no out-of-the-box method available that is able to capture all data sent between ROS nodes. In this chapter, multiple methods of capturing the data will be considered, each will be discussed below.

5.1 Instrumenting ROS nodes

Instrumenting the code is an option i.e. adding code to the existing ROS code which would allow one to capture all messages sent. This option is used by ROSMonitoring as can be seen in the previous chapter. ROSMonitoring is not the only solution using instrumentation for runtime verification, in the paper: “Instrumentation of intermediate code for runtime verification” Gates et al(2003) [13] introduce the idea of using code instrumentation to verify programs at runtime. Gates et al use code instrumentation to monitor certain variables of the system, unlike ROSMonitoring which uses code instrumentation to monitor messages sent between two nodes.

To monitor ROS messages, at every place in the code where data is being sent, code should be added. The code that is added would for example send identical messages to a “Collector” node which converts it to ComMA traces and will then send it to the online monitor.

Instrumenting the code could be done manually but this would drastically increase the time it takes to implement a ROS Node. This would namely result in the addition of code at every place where a message is sent.

Instrumenting the code automatically is another option, this is however a lot more complex and other disadvantages of instrumenting will remain. The most prevalent disadvantage is the fact that each message will be sent twice which could lead to (network) performance issues.

Figure 6 has a general overview of the data flow when instrumentation, as described above, would be used. ROS Node A would send the message: “Hello” to Node B. Because of the instrumentation, this message is also sent to the Monitoring node. The message is thus sent twice.

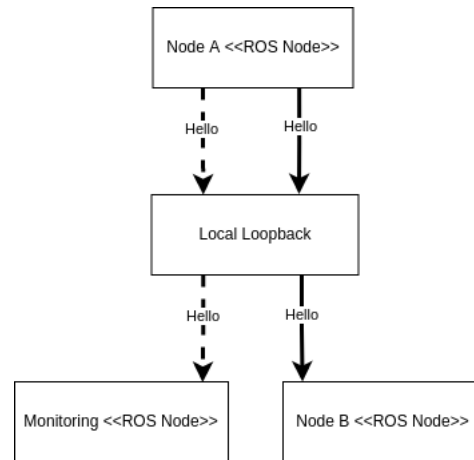


Figure 6: Data flow for a single message when instrumentation is used.

5.2 ROS APIs

ROS 2 uses a DDS (Data Distribution Service) [14] in combination with RTPS [15] as a communication method between nodes. In the previous version of ROS, ROS 1, ZeroMQ [16] and ProtoBuf [17] were used to communicate between ROS nodes. The usage of DDS in ROS2 allows the developer to specify a Quality of Service which has to be upheld.

At the moment, ROS 2 does not support monitoring for all available communication methods. Topics can be listened to, by using `rclcpp` or `rclpy`. Services and Actions on the other hand, cannot be listened to via ROS APIs. This is most likely a result of the fact that topics have a one-to-many relationship, if we want to listen to the topic we can simply subscribe to the topic and receive any message sent on the topic. Services and actions have a one-to-one relationship which does not allow one to subscribe to these messages.

5.3 Eavesdropping messages

Eavesdropping messages is another option. As previously stated, the DDS uses RTPS as a low-level communication protocol. The case study was configured to specifically use the eProsima [18] while others DDSes can be used as well. If the ROS nodes run on the same system, shared memory can also be used as communication method. RTPS is based on the TCP/UDP/IP protocols. This allows tools such as `tcpdump` [19] and `Wireshark` [20] to capture the data. These tools allow one to listen to and analyze the messages being sent and received via network interfaces. `Wireshark` has support for RTPS, which allows us to easily dissect these messages. `Wireshark` itself is a program with a graphical user interface but also has a terminal version, `TShark`[21]. `TShark` would allow us to more easily integrated it with other programs as the standard output can easily be captured and parsed by another program. Communication via shared memory has to be disabled to be able to capture the data with `TShark`.

An example of the capture of data can be found in Figure 7, Node *A* sends a message saying: “Hello” to Node *B*. Starting at Node *A* the message goes through the Local Loopback to Node *B*. The monitor is able to eavesdrop this message by using `TShark`, even though the message was not meant for the monitor. The main advantage of this method is that it is programming language independent, this method uses the standardized way ROS messages are sent between nodes. Regardless of the language used to implement the nodes, the monitor node can listen to their communication without making changes to the code of the nodes.

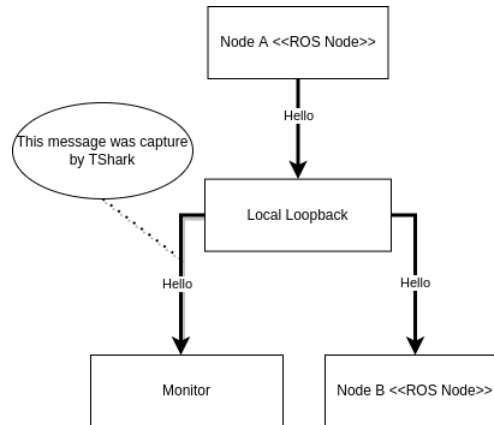


Figure 7: Data flow for a single message when eavesdropping is used.

5.4 Intercepting all messages

Intercepting all messages by adding a new layer to the middleware is another option to capture data. One of the existing solutions, ROSRV, uses this method. A new layer was added to the middleware besides the, in ROS 1 used, ROS Master. In ROS 2, ROS master is replaced by a DDS. Developers are free to choose from a set of DDS vendors. Each DDS uses RTPS as protocol, the implementation, however, is different for each of them. This extra layer would require different implementations for each of the different DDSes. Perhaps an even greater challenge is the fact that the usage of a DDS allows one to specify a QoS (Quality of Service). The extra layer would need to adhere to this same QoS as the DDS does.

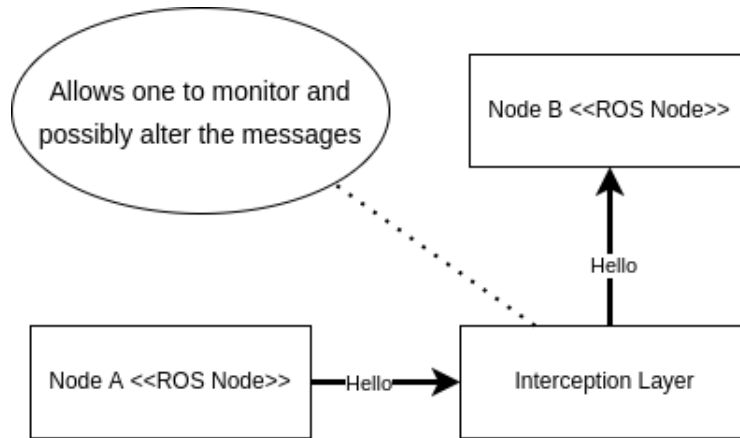


Figure 8: Data flow for a single message when intercepting messages

5.5 Conclusion

After considering the different possibilities to capture the data, eavesdropping the messages using TShark seems to be the most optimal solution at this moment. Eavesdropping message has a number of limitations, firstly it will not be possible to edit messages, in addition, it will not be possible to stop messages from ever reaching its goal. This is however not a problem as ComMA is currently not able to suggest that message should not be sent or if the message should be altered. If we compare eavesdropping messages with instrumenting nodes, then the biggest advantage of eavesdropping messages is, that it is not dependent on the language in which a ROS node is written.

6 Capturing message - implementation

As previously explained, eavesdropping messages using TShark has been chosen as the method to capture mutual messages between ROS nodes. As TShark captures any packets sent on the selected interface we will need to filter the captured packets. TShark's output should only contain RTPS packets needed for the online monitoring, at the moment the only packets needed are packets containing messages sent between ROS nodes.

To capture only the messages send by ROS nodes a number of arguments are supplied to TShark, these arguments and an explanation can be found in Appendix B.

A more detailed explanation of TShark can be found in TShark's manual [22]. Using TShark with the arguments found in Appendix B, only packets containing ROS messages are captured. An example of a captured packet can be found in Appendix C. This captured packet cannot be used immediately. The `rtps.issueData` field contains the actual data sent between ROS nodes. This data is still serialized as can be seen in the example below:

```
"rtps.issueData": [  
  "0000000005000000040000006f6e65000400000074776f493020482903fe94"  
]
```

After the `rtps.issueData` has been captured it still needs to be deserialized, the next section discusses this in more detail. TShark outputs all packets to the standard output, to capture this a program called ROS2ComMA has been created. In Figure 9 the general architecture can be seen. TShark captures the RTPS packets sent over the local loopback and outputs these to the standard output. ROS2ComMA can then capture TShark's output and process it.

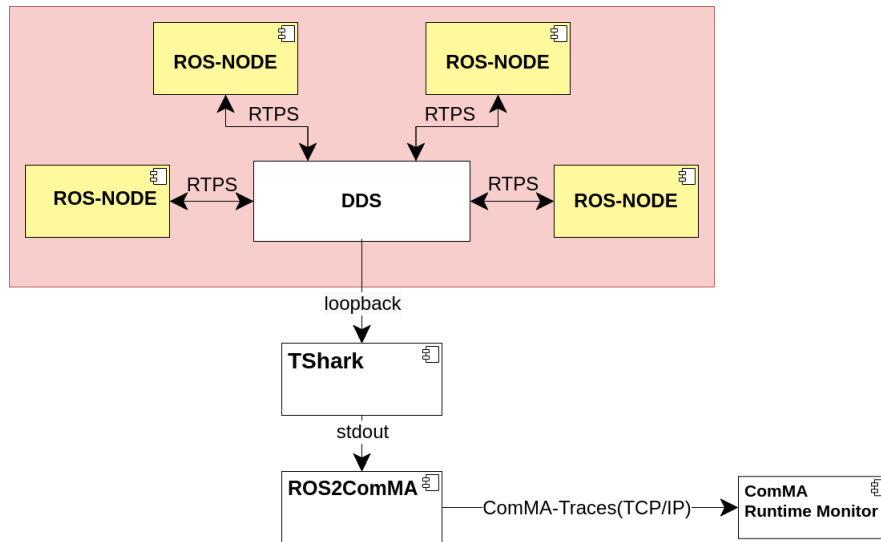


Figure 9: Flow from ROS to ROS2ComMA

6.1 Deserializing the captured data

In order to efficiently send data between ROS nodes, all messages sent between nodes are serialized. ROS Messages are defined in IDL(Interface definition language) [23]. The message definition is used to generate Python and C++ code at **compile-time**. This allows one to use these messages in their own code, the generated code will take care of the serialization and deserialization.

Last section we concluded that the data captured by TShark cannot immediately be used after being captured. Before the data can be used, it has to be deserialized, this will be done by ROS2ComMA. An ideal solution would be one that is able to deserialize any message with as little changes as possible when new messages are added. To achieve this, `ros_idl_runtime_py` [24] and `rcipy` [25] will be used.

6.1.1 `rosidl_runtime_py`

`rosidl_runtime_py` is a package that allows one to get the data structure of a message at **runtime**. This allows us to create a generic program that is able to deserialize any message at runtime. The data structure of a message can be retrieved using only the name of the message sent. This name can easily be retrieved from the packet captured by TShark. In the example packet that can be found in Appendix C, the following line can be found:

```
[typeName: machine_interfaces::srv::dds_::ButtonClick_Request_]
```

This field called: *typeName* cannot directly be used, it needs some modification before being used with `rosidl_runtime_py`. It needs to be modified to the following form:

```
machine_interfaces/srv/ButtonClick
```

Supplying this modified *typeName* to the `get_message()`, `get_service()` or `get_action()` function of `rosidl_runtime_py` will yield the data structure of the message.

6.1.2 `rcipy`

`rosidl_runtime_py` allowed us to retrieve the data structure of the data received, what is left is actually deserializing the message. This is what the library `rcipy` will be used for. The serialized data can be deserialized using the method: `rcipy.serialization.deserialize_message()`. The function expects two arguments:

- The serialized data, in our case this is the `rtps.issueData` as mentioned in last section.
- The data structure, in our case the data structure obtained with `rosidl_runtime_py`.

7 Generating ComMA traces from ROS data

In the previous chapter, we discussed the capture of the data sent between the different ROS nodes. Before this data can be sent to the runtime monitor of ComMA, ROS2ComMA needs to convert this data to ComMA traces. ComMA traces are events that are happening in the system under monitoring. A trace contains, among other things, the event kind(Command, Signal, Notification or Reply), timestamp, source, destination and parameters of the event. Section 7.6 discusses the traces in more detail. As previously stated communication between ROS nodes is done using three patterns: Topic, Service or Action. ComMA however does not have these exact same patterns. This is why a conversion has to be made between the patterns in ROS and the patterns available in ComMA. The patterns in ComMA are the following [2]:

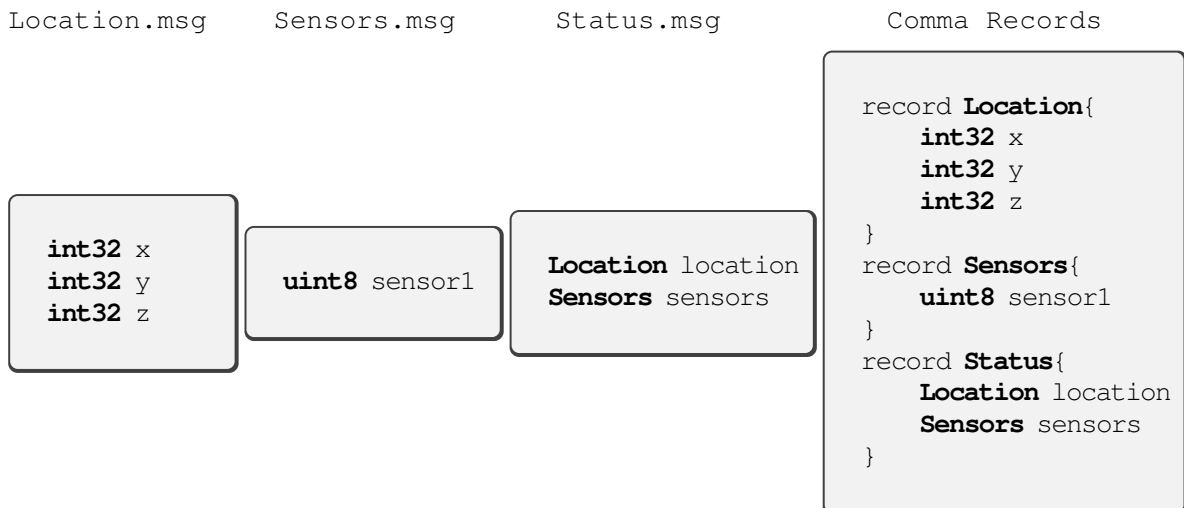
- **Commands:** synchronous input calls that require a reply possibly with a value. The caller is blocked until the reply is received.
- **Signals:** asynchronous input calls, no reply is allowed. The caller is not blocked.
- **Notifications:** asynchronous output - not blocking.

7.1 Primitive datatypes

Both ROS and ComMA have their own set of primitive datatypes, unfortunately, they do not share the same exact types which requires some conversion and possibly. In Appendix D the conversion between ROS' primitive datatypes and ComMA's datatypes can be found. Most conversions can be made without the loss of information. ROS has a wide variety of integers whereas ComMA by default only has one, namely int. It is, however, possible to create new types, an int8 can for example be created with the following line: "type int8 based on int". The resulting int8 in ComMA is however not constrained to the minimum and maximum value possible with 8 bits. This thus results in the ComMA int8 being able to be assigned values not possible in ROS' int8. All integers but one can be defined this way. The uint64 cannot be implemented as the int datatype of ComMA is a signed 64-bit integer. ROS messages that are monitored can thus no longer use the uint64 data type. ComMA does have type mappings which allow one to map types to types of a certain language. This might be a solutions to this problem, note that this has not been tested.

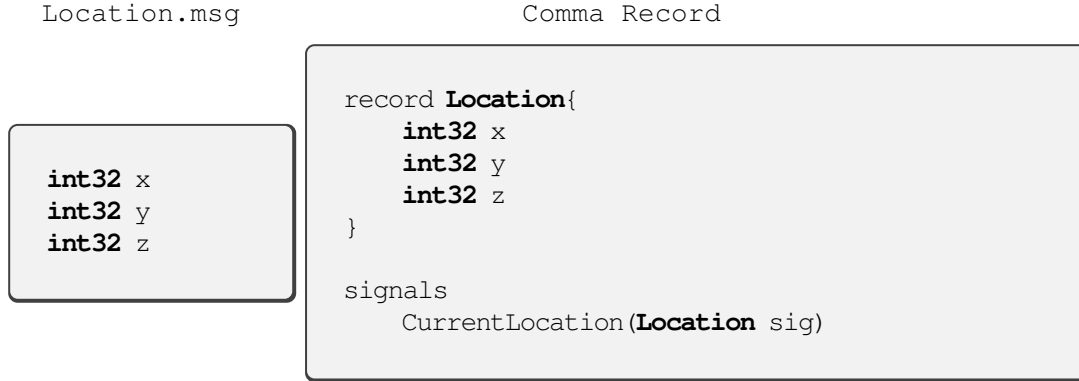
7.2 Nested messages

Both ROS and ComMA support nested messages/records. Below an example can be found which shows how a nested message in ROS should be defined in ComMA.



7.3 Translating Topics

Translating Topics to ComMA is done by the usage of ComMA's signal, the concept between ROS' Topic is equal to ComMA's signal. Both are asynchronous and non-blocking messages. An example can be seen below, in this example, a Location message is being published on the topic: *CurrentLocation*.



The ComMA signal *CurrentLocation* could have been implemented using 3 different arguments instead of a single record. A record containing 3 fields has been chosen instead. This solution has been chosen to maintain the name of the message: *Location* and to more easily implement nested messages.

7.4 Translating Services

Translating Services to ComMA is done by using ComMA's commands. ROS Services can be implemented as both synchronous and asynchronous in Python and asynchronous in C++. If the synchronous implementation is used, nothing can be done on the caller's side until a response has been received [26]. If the asynchronous implementation is used, the ROS node does not need to wait until a response has been received.

ComMA's **commands**, on the other hand, are synchronous and blocking on the caller's side. This results in some limitations.

In Figure 10 an example of a ROS node making asynchronous service calls can be seen. Node A requests the **red** service from Node B. Before Node B sends a response, Node A requests the **blue** service from Node C. This is would not possible with ComMA's command if both commands are part of the same interface. To prevent this the **red** service and **blue** service should be in separate interfaces, this can be defined in the configuration file. An example of a configuration file that can be supplied to ROS2ComMA can be found in Appendix C

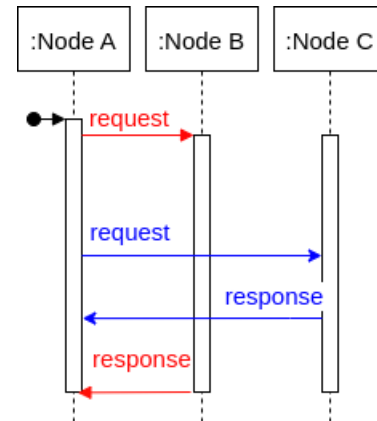
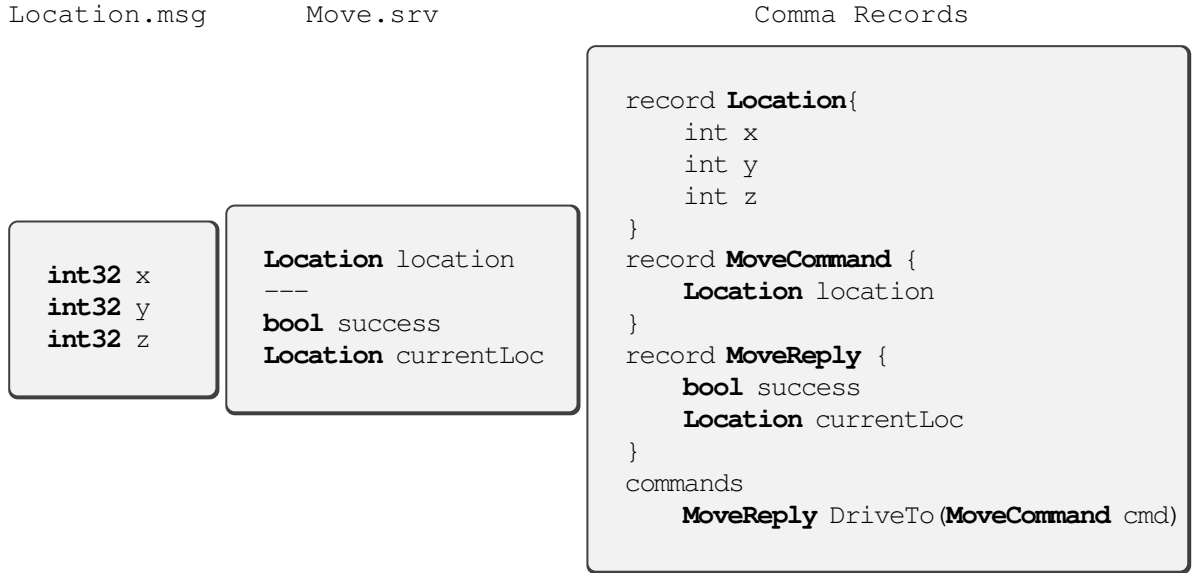


Figure 10: Asynchronous calls

An example of the conversion of a ROS service to ComMA's command can be found below. In this example, a service called "DriveTo" is offered. This service is defined by the file "Move.srv". Note that the request of the service contains a custom type, defined in Location.msg. This will result in an extra record called Location.



7.5 Translating Actions

The case study used in the research internship did not make use of ROS actions. Because of this, the conversion has not been implemented. The conversion of ROS Actions to ComMA is fairly similar to the conversion of ROS Services to ComMA.

7.6 Generating the traces

In the sections above the conversion of the data sent by ROS nodes was done. ComMA however requires more than just the data to be sent. ComMA has two formats in which events can be defined, during this project the JSON-events [27] have been used. Each event is in the JSON-format with the following fields: kind, timeStamp, interface, source, destination, sourcePort, destinationPort, method and parameters. For each of these fields, a value is derived from the sniffed ROS packets.

- **kind**, the kind of the event can be a Command, Reply, Signal or Notification. Messages published on a ROS topic will have Signal as its kind. When a message is part of a ROS service it is either a Command or a Reply.
- **timeStamp**, the timestamp of an event denotes the time at which the message was sent, this timestamp is available in the sniffed packet. Note that the timestamp is not the time at which the packet was sniffed but the actual time the message was sent by the ROS node.
- **interface**, the interface of an event should be the name of the interface model of which the message is part. ROS, however, does not have something similar to ComMA's interfaces. In the configuration file that can be found in Appendix E, one can specify which topics, services and/or actions are part of which interface.

- **source**, the source field of an event is equal to the source from which the ROS message originated. An ideal solution would be one where the source value has the name of the node. Each ROS node has a unique name that can be defined in the code or via the command line. At the moment no solution has been found to retrieve this unique name from a sniffed packet. The source is now defined in the configuration file. Based on the interface and method, both of which can be extracted from the sniffed packet, the value entered as source can be retrieved from the configuration file.
- **destination**, similar to the source of an event, the destination of an event is retrieved from the configuration file.
- **sourcePort and destinationPort**, in the current case study the sourcePort and destinationPort were not necessary as each interface was only used once. This is why the sourcePort and the destinationPort are equal to the value of the interface plus a trailing client/server. If the interface would be called “ITest” the sourcePort would be: “ITest-Client” and the destinationPort: “ITestServer”. While this was sufficient for the current case study it might be the case that for other systems this implementation will no longer suffice.
- **method**, the method of an event in ComMA is the name of the Command, Notification or Signal. The name of the ROS Topic, Service or Action is used as method.
- **parameters**, the parameters field of an event is the field in which the actual data is put. In our case, the data is equal to the content of the sniffed ROS message.

8 Online monitor feedback

As previously mentioned, ComMA’s online monitor is in the early stages of its development. Because of this, at the beginning of the research internship, the online monitor was not capable of producing feedback at runtime while it was able to receive the messages at runtime. It was in fact a glorified offline-monitor able to receive input at runtime.

8.1 What should happen when the monitor finds non-conformance?

Offline monitors are relatively limited, they are able to check the traces it has been given and output errors or warnings that occurred. Online monitors on the other hand have more options. Huang et al. [12] allowed the online monitor of ROSRV to be able to actually intervene when safety policies were violated. This was possible as the online monitor was a layer which all messages had to go through which allowed the online monitor to alter the messages before sending them to the actual destination.

This can easily be implemented for simple safety policies such as a maximum speed of a motor. A message can easily be changed if it tries to set the speed to 5 m/s when the safety policy defines that the maximum speed is 4.5 m/s. However, a question that comes up is: should the original sender of the message also be made aware of this change? The original sender of the message might be depended on the actual speed. If it is not made aware that the actual speed is set to 4.5 m/s it might make incorrect calculations based on the presumed speed of 5m/s. This problem is currently not tackled by ROSRV and is not easily solved as this problem highly depends on the actual implementation.

Besides policies/constraints on single values, ComMA is also able to monitor time constraints, interfaces, components and ports. A state-machine can for example be defined, if a transition caused by an event is not possible in the defined state-machine an interface error is thrown. This only poses more complicated questions on what to do when non-conformance is found. ComMA’s offline monitor currently has interface/component/port errors and warnings. When the offline monitor encounters an error the monitoring is stopped for a specific part of the system. When an interface error is encountered this connection will no longer be monitored. Component errors will result in a functional component constraint to no longer be monitored and port errors will result in function/time/data constraints to no longer be monitored [2].

Online monitoring would allow one to, in theory, recover from some of these problems. If the online monitor detects that an illegal state is reached, the monitor might be able to provide useful feedback. With this feedback, the system could possibly enter a recovery state in which it makes small changes that allow the system to get back into a legal state. This would require specific code to be written to deal with the feedback of the monitor and to recover from certain errors. Not all errors can or should be recovered from, it might be safer to safely shut down instead of trying to recover from an error.

8.2 Performance of the online monitor

Something introduced by online monitoring is the dependency on the performance and timeliness of the monitor. This is less of a problem for offline monitoring as it can very well be run during the night and can thus take several hours to finish. For online monitoring, however, you want to have the results as fast as possible. Finding out that a constraint is violated a minute after it had occurred might be too late in some cases.

Whether the online monitor is able to process the input fast enough depends on a number of factors. This for example includes the size of the ComMA model, the number of constraints specified and the number of traces sent each second. In the earliest version of the online monitor, a new event would only be accepted if the monitor had processed the previous event. This resulted in an accumulation of traces that were not processed yet, these traces had not yet reached the online monitor. The online monitor was thus not aware if it was lagging behind and if so, it wasn't sure how much it was lagging behind. This is why a queue within the online monitor is introduced, as can be seen in Figure 11. Traces sent to the runtime monitor will be put in the queue immediately after receiving them. The online monitor can then retrieve the event from the queue one by one. The online monitor could eventually use this queue to see how many traces it is lagging behind. An indication of the time it is lagging behind could also be made, this could be done in two ways. The first is to calculate the difference in time between the event currently being processed and the last event received. And secondly, the difference in time between the current time and the timestamp of the event being processed. As ROS Nodes can run on different systems, the latter would require that the time of the ROS nodes is synchronized with the time of the system ROS2ComMA is running on. This could then be sent as feedback via the same output socket as earlier. This allows any program listening to the socket to know that the online monitor has a backlog of traces.

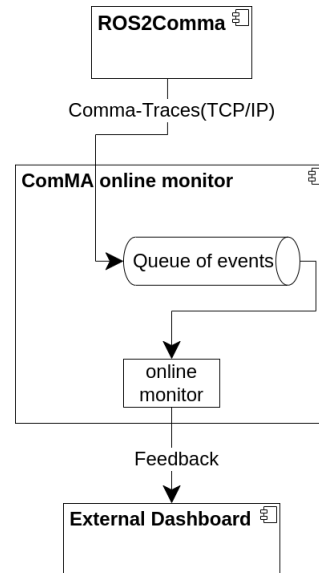


Figure 11: Initial online monitor's feedback design

8.3 What should the feedback look like?

Receiving feedback from ComMA's online monitor at runtime is needed to actually see the added value of an online monitor. This is why the following proposal has been made.

All feedback from the online monitor should, initially, only be sent to a single location. It should thus not be sent to the node(s) involved in the trace that caused the feedback. To allow this to be implemented, a secondary output socket should exist besides the already existing input socket. This output socket can then be used by a separate program or ROS node to interpret the feedback.

The feedback that should be output by the online monitor could be of the form that is currently found in the output file: `taskResult.json`. An example can be seen below:

```
"warnings": [  
  {  
    "ruleName": "Frequency_update",  
    "errorMessage": "Event is not observed in the expected interval.",  
    "umlFile": "$PATH_TO_UML_DIAGRAM$"  
  },  
  {  
    "ruleName": "NORTH_MAX",  
    "errorMessage": "Data constraint is violated.",  
    "umlFile": "$PATH_TO_UML_DIAGRAM$"  
  }  
]
```

The output above does not contain any specifics as these are all included in the PlantUML diagram. The runtime monitor does not generate the PlantUML files at runtime, the information available in the PlantUML should thus be included in the feedback sent by the online monitor. This is probably the easiest by returning the JSON trace(s) that caused the constraint violation. This trace will contain data such as the timestamp, source, destination, method and interface. Another useful addition is that the specifics of a violated constraint are sent. For time constraints this could for example be that the maximum interval between two messages is set to 5ms but the last interval was 7ms.

As mentioned earlier, ComMA's offline monitor is capable of generating errors and warnings for interfaces, components and ports. Time and Data constraints warnings should be sent immediately after a constraint is violated, the online monitor will continue monitoring after a time or data constraint has been violated. Other errors should be sent immediately after occurring but might, unlike the constraint warnings, result in the online monitor being stopped.

9 ComMA model for the case study

The case study that was mentioned earlier has been used to create a ComMA model. Using this ComMA model and ROS2ComMA the case study could be monitored. To monitor the case study, the code was analyzed. This made it possible to create a Component, Interfaces and time/data constraints for the interfaces.

9.1 Component: Traxxas

The case study used did not include a clearly defined state machine. A small one could be extracted by examining the code, this state machine can be seen in Figure 12. The state machine is not as large or complex as ComMA is capable of monitoring but it does show that a state machine implemented by one or more ROS nodes can be monitored by ComMA. The state machine is defined in a component as it is influenced by multiple interfaces.

```
initial state Prepare{
  signal ILocationLocalServer::location_local(loc)
  next state: Prepare
  signal IEnemyServer::enemy_detector(enemy)
  next state: Driving
}

state Driving{
  signal ILocationLocalServer::location_local(loc)
  next state: Driving

  signal IWaypointServer::next_waypoint(waypoint)
  goal_x := waypoint.north
  goal_y := waypoint.east
  next state: Surveiling
}

state Surveiling {
  signal ILocationLocalServer::location_local(loc)
  current_x := loc.north
  current_y := loc.east
  next state: Surveiling

  signal IEnemyServer::enemy_detector(enemy)
  where abs(current_x - goal_x) < max_distance
  and abs(current_y-goal_y) < max_distance
  next state: Driving
}
```

Figure 12: ComMA implementation of the Traxxas Component.

The state machine defines that the UGV requires an `enemy_detector` message to be sent before it starts driving. In the state `Driving`, the UGV can start driving and calculate a waypoint to drive towards. Once it has calculated a waypoint a transition has to be made to the state `Surveiling`. Before this transition is made, the ComMA model defines that the `goal_x` and `goal_y` values of this waypoint are saved. In the state `surveilling` the current location is saved every time a `location_local` message is received. If an `enemy_detector` message is received a calculation is made to check whether the UGV is close enough to the waypoint to correctly check for an enemy. If this was the case a transition is made to the `Driving` state.

9.2 Interface: ITraxxasControl

From the code a couple timing constraints can be derived. The rate at which the `set_steering_angle` and `set_throttle` services are used is defined by the following line of code which is located in the `control_converter` node:

```
publish_signal_timer_ = self.create_timer(0.02, publish_signal)
```

This timer makes sure that the `publish_signal` is called every 20 milliseconds i.e. at a rate of 50hz.

In ComMA, two timing constraints are created to be able to monitor the rate at which these messages are sent. The implementation of these timing constraints can be seen in Figure 13. It is of course practically impossible for the messages to, always arrive with an interval of exactly 20.0 milliseconds, this is why the maximum interval is set at 20.1 instead of 20.0 milliseconds.

timing constraints

```
SetSteering_rate  
signal set_steering_angle - [ .. 20.1 ms ] -> signal set_steering_angle  
  
SetThrottle_rate  
signal set_throttle - [ .. 20.1 ms ] -> signal set_throttle
```

Figure 13: ComMA implementation of timing constraints for throttle and steering.

9.3 Interface: ILocation

Similar to the previous interface, the rate at which the `location_location` is sent is defined by the following line of code which is located in the `telemetry_converter` node:

```
publish_signal_timer_ = self.create_timer(0.02, publish_signal)
```

Monitoring this publishing rate is done by the following time constraint:

There are also some data constraints that can be implemented for the `ILocation` interface. The grid in which the UGV is driving is defined in the `learning_node` by the following line of code:

```
//Max interval between two LocationLocal signals
Frequency_update
signal location_local - [ .. 20.1 ms ] -> signal location_local
```

Figure 14: ComMA implementation of timing constraints for the location update.

```
Grid([0, 10, 20, 30, 40, 50, 60, 70], [0, 10, 20, 30, 40, 50, 60, 70])
```

From this, it can be concluded that the UGV should always be at a location(North,East) for which it holds that $0 \leq North \leq 70$ and $0 \leq East \leq 70$. To monitor that the UGV never goes beyond (0,0) and (70,70) the following data constraints are defined in ComMA:

```
data constraints
variables
LocationLocalSignal location

MIN_MAX_LOCATION_NORTH
signal location_local(location) where location.north <= 70.0
and location.north >= 0.0

observe
received := location

MIN_MAX_LOCATION_EAST
signal location_local(location) where location.east <= 70.0
and location.east >= 0.0

observe
received := location
```

Figure 15: ComMA implementation of data constraints for the UGV's location.

9.4 Interface: IWaypoint

For the IWaypoint interface, similar data constraints can be implemented. As previously mentioned in the section about ILocation, the grid in which the UGV is driving is 70 by 70. The learning_node which determines the next waypoint to discover should never send the UGV to a waypoint outside of the grid. To monitor this, the following ComMA data constraints are implemented:

```
data constraints
variables
LocationLocalSignal waypoint

MIN_MAX_WAYPOINT_NORTH
signal next_waypoint(waypoint) where waypoint.north <= 70.0
                                and waypoint.north >= 0.0

observe
received := waypoint

MIN_MAX_WAYPOINT_EAST
signal next_waypoint(waypoint) where waypoint.east <= 70.0
                                and waypoint.east >= 0.0

observe
received := waypoint
```

Figure 16: ComMA implementation of data constraints for the waypoint.

10 Known issues and Limitations

During the research and implementation phase problems and limitations surfaced. Throughout the internship many of these problems were solved, there are however some problems that have not been solved.

10.1 Order of messages

ComMA relies on the correct order of messages being sent. ROS' DDS(Data Distribution Service), however, does not guarantee the order of messages. At the moment there is no QoS in ROS regarding the order of messages. It could thus be the case that messages *A* with timestamp 5 is sent before message *B* with timestamp 4.

Possible solution:

Messages could be sent with a delay of for example X ms. This is to make sure the order being sent to ComMA is guaranteed if messages do not arrive over X milliseconds late. This would, however, delay the detection of faults, it will however also prevent some false-positives.

example:

Message 1 with timestamp 01:10:00:40 is received at 01:10:00:42

Message 2 with timestamp 01:10:00:38 is received at 01:10:00:43

The specification requires message 2 to happen before message 1. Looking at the timestamps this is the case, as 01:10:00:38 is before 01:10:00:40, if we look at the time the messages have been received this is no longer the case. As ROS does not have a QoS regarding the ordering of messages the example above might occur which would result in a false positive. If a delay is implemented, we for example wait X milliseconds before sending message 1. Within those X milliseconds message 2 is received, comparing the timestamps of the two messages allows us to correct the order before sending it to ComMA.

It could of course occur that messages arrive $X + 1$ milliseconds late. In this case, ComMA would receive the message in the wrong order. It is however the question if you even want to check the messages if messages are $X + 1$ milliseconds late. One could ask the question: is the system still reliable if messages arrive later than the specified amount of "waiting time"?

10.2 Limited to Eprosima DDS

ROS 2 has the option to choose between different DDS vendors. Each DDS vendor's implementation of the RTPS protocol is slightly different. It is thus not possible to use any other DDS besides the Eprosima DDS. The Eprosima DDS was chosen as this DDS was required by the case study.

11 Evaluation

11.1 Activities

My research mainly focused on the research question RQ-1: How can ComMA’s runtime monitoring be applied to ROS? The research included both literature research and implementation of a POC (Proof Of Concept) in which the behavior of ROS nodes can be monitored using ComMA’s online monitor.

A start has recently been made on the implementation of the online/runtime monitor of ComMA. Before my research internship, there had not been any substantial projects that used the online monitor. The online monitor is based on the offline monitor of ComMA, which unlike the online monitor is already used a lot. My research has contributed to the development of the online monitor by testing the current implementation. During my internship, I have reported a couple of bugs, suggested ways to improve the online monitor, and overall gave feedback on the current state of the online monitor. The bugs have now been fixed and even some of the suggested ways to improve the online monitor have been implemented. These bugs and improvements will be discussed in more detail in the next section: “Results”. Besides the improvement of the online monitor, I have also been able to implement a working Proof Of Concept that is able to monitor the case study described in Section 3.

11.1.1 Familiarizing myself with ComMA and ROS

In the beginning of the internship, there were two main unknowns, namely: ComMA and ROS. I had global knowledge of ComMA and the main idea behind it. But I was not aware of the specific possibilities and the way it could be used. The second unknown, was ROS, or rather ROS 2. I had previously worked with ROS 1 but had never worked with ROS 2. In the beginning, the main question was: How can communication between ROS nodes be captured, converted and sent to the ComMA runtime-monitor?

Having two unknowns, a choice had to be made: which unknown to explore first? I chose to first get acquainted with ComMA. The capabilities of ComMA would ultimately influence the research into the monitoring of ROS nodes. To familiarize myself with ComMA, I followed the tutorial [28] which gave me a more clear understanding of ComMA.

After that, I looked into ROS 2 and its differences with ROS 1. It turned out that the main “philosophy” had not changed and the main changes were under the hood regarding the communication between ROS nodes. Changes under the hood are normally not important to the end-user, as long as the end-user cannot notice the differences. These changes were however important for me, to be able to monitor the behavior of the ROS nodes, the mutual communication between these ROS nodes had to be caught.

11.1.2 Answering the first two sub-questions

Discovering that the underlying communication was changed in ROS 2 resulted in me answering the first two sub-questions, RQ-1.1: Are there any existing online monitoring solutions for ROS? and RQ-1.2: How can data sent between ROS nodes be captured?. Answering the first question could help me with answering the second question. If a runtime monitor for ROS would exist, it would need to somehow get information from the ROS nodes to the online monitor which is what I needed to do as well.

It turned out that two runtime-monitor solutions existed, albeit for ROS 1. The first was ROSRV [12], ROSRV captured the mutual communication between ROS nodes by introducing a new

layer that each message had to go through. ROSMonitoring [9] was the second solution I found. ROSMonitoring captured the mutual communication between ROS nodes by instrumenting each ROS node to send each message twice, once to the recipient, and once to the online monitor. A small summary of these solutions can be found in Section 4.

Answering the question regarding the existing online monitoring solutions for ROS 1 gave me two methods to capture the data of ROS nodes. Of course, these methods were not guaranteed to work for ROS 2, but they did give me some inspiration for what could be possible. A third solution was mentioned by Mathijs Schuts in one of our meetings. During this meeting, he mentioned that at Philips, Wireshark [20] was used to capture data sent between two or more programs to be able to eventually generate ComMA traces. In Section 5 a comparison between these three solutions has been formulated. Capturing the messages using TShark [21] turned out to be the best solution, in this same chapter, the actual implementation and the design choices are explained as well.

After being able to capture the data the next step was to generate traces that ComMA could use to verify the case study. Both ROS and ComMA have their own set of data types, communication patterns, etc. A simple conversion between ROS messages and ComMA traces was thus not possible. For each different data type and communication pattern, a different conversion had to be made. In Section 7, RQ-1.3: How can ComMA-formatted traces be generated from the data of ROS nodes? is answered. Unfortunately, some conversions are currently not possible due to non-supported data types by ComMA, an example of such a data type is an unsigned 64-bit integer. During the implementation of the conversion, the software was tested using the earlier described case study.

11.1.3 Implementing the ComMA model for the case study.

After being able to capture and process the data sent by ROS nodes, the next step was to implement the ComMA model. In Section 9 a description of the ComMA model used to monitor the case study can be found. The ComMA model includes a component which uses different interfaces. For each interface, time and data constraints are defined if applicable.

11.2 Results

During the research internship, multiple results have been accomplished. The main result of the internship is this document and the actual implementation of the POC. The implementation of the POC includes the following:

- ROS2ComMA, a program written by me which is able to eavesdrop ROS messages, convert these to ComMA traces and finally send them to the online monitor.
- FeedbackDashboard, a simple dashboard that is able to interpret and show the feedback sent by ComMA's online monitor.
- A ComMA project that generates an online monitor that can be used to monitor the case study mentioned earlier.

11.2.1 Improving ComMA's online monitor

Besides the results mentioned above, I also managed to contribute to the improvement of ComMA's online monitor. During the usage of the online monitor, I encountered a couple of bugs and shortcomings. While testing an initial implementation of the conversion from ROS messages to ComMA traces, I found that the warnings/errors reported, were rather vague when records

were used. When data constraints were violated the error message would simply state that a record of a certain type violated the data constraint. When data constraints were violated by primitive datatypes the values were reported in the error/warnings. In Appendix F an example of ComMA’s output can be seen when using records. In the part, outlined in red, it can be seen that no information about records is given. When using offline-monitoring this is not a problem as the trace causing the warning is part of the input file and can thus easily be back-traced. For online monitoring, however, traces are sent via sockets and thus not automatically saved. After I reported this, more information about records has been added as can be seen in Appendix G. Later on, the possibility to save all traces sent to the online monitor was added as well.

11.2.2 Incorrect calculation of time constraints

One of the bugs I found during the internship was an incorrect calculation of time constraints when using the JSON-traces [27]. ComMA has two types of traces, ComMA-traces [29] and JSON-traces [27]. They have the same set of features but are constructed differently. While testing my work using the JSON-traces as input for the online monitor, I discovered that the time constraint did not function as expected and the time-related statistics did not either.

11.2.3 Improvement of the online monitor

As I mentioned earlier the online monitor is in the early stages of development. During the usage of the online monitor I found that, in the current state, it was not an actual online monitor. The monitor would only calculate and output the results once the “input-socket” was closed, after the last trace had been sent. This resulted in the online monitor being nothing more than a glorified offline-monitor, capable of receiving traces one at a time(at runtime) rather than reading all traces from a file afterward. To make the online monitor an actual online monitor I worked out a design with, what I thought would be useful improvements. I have documented these improvements in Section 8. I presented my improvements in a meeting with Ivan Kurtev. Ivan has since been able to implement the first set of improvements that I suggested. The latest version of the online monitor can immediately check whether a trace is in conformance with the model defined in ComMA. If it is not in conformance with the model, feedback is sent over a “feedback”-port. This feedback includes details on why the trace sent is not in conformance with the model.

11.3 Reflection & Evaluation

I would personally say that the research internship went well. The main questions with which the internship started have been answered which was of course the main goal. Besides that, I was very pleased that I was able to contribute to the improvement of the actual online monitor.

11.3.1 Meetings

Besides being able to successfully achieve these results, the process that led to these results went fairly well too. What I think went well were the weekly meetings with Jozef Hooman, the company supervisor. Each Thursday morning we discussed the current progress, topics I would be looking into the following week and discuss possible new angles of the research during the internship. Having these weekly meetings helped me work an equal amount of hours on the internship each week. Something many students, including me, “suffer” from is procrastination. It has occurred more than once that assignments for a course would be implemented in the last couple of weeks instead of throughout the course. Having these weekly meetings helped me maintain gradual progress. The weekly meetings “forced” me to put in the time, I could of course not show up at a meeting without any results. I would not go as far as saying that I would not have achieved the same final results if I had not put in the same amount of time in the internship each week. It is however clear that it is not an ideal situation if most of the work is done at the end of an internship.

During these meetings, I discovered that there was still room for improvement. Especially in the early stages of the internship, I had a lot of questions regarding ComMA, and the internship in general. Remembering these questions, without writing them down, is of course not easy. As I did not always write these questions down it would sometimes occur that I forgot to ask some of these questions during the weekly meetings. To prevent this, I started to prepare myself better for these meetings. Throughout the week I wrote down any questions I had in a document for the next meeting. Using this document I was able to prepare myself before the meeting by reading through the document and remembering the questions. At the end of the meeting, I would quickly go through the document one last time to make sure I had not forgotten to ask any questions.

11.3.2 Making notations

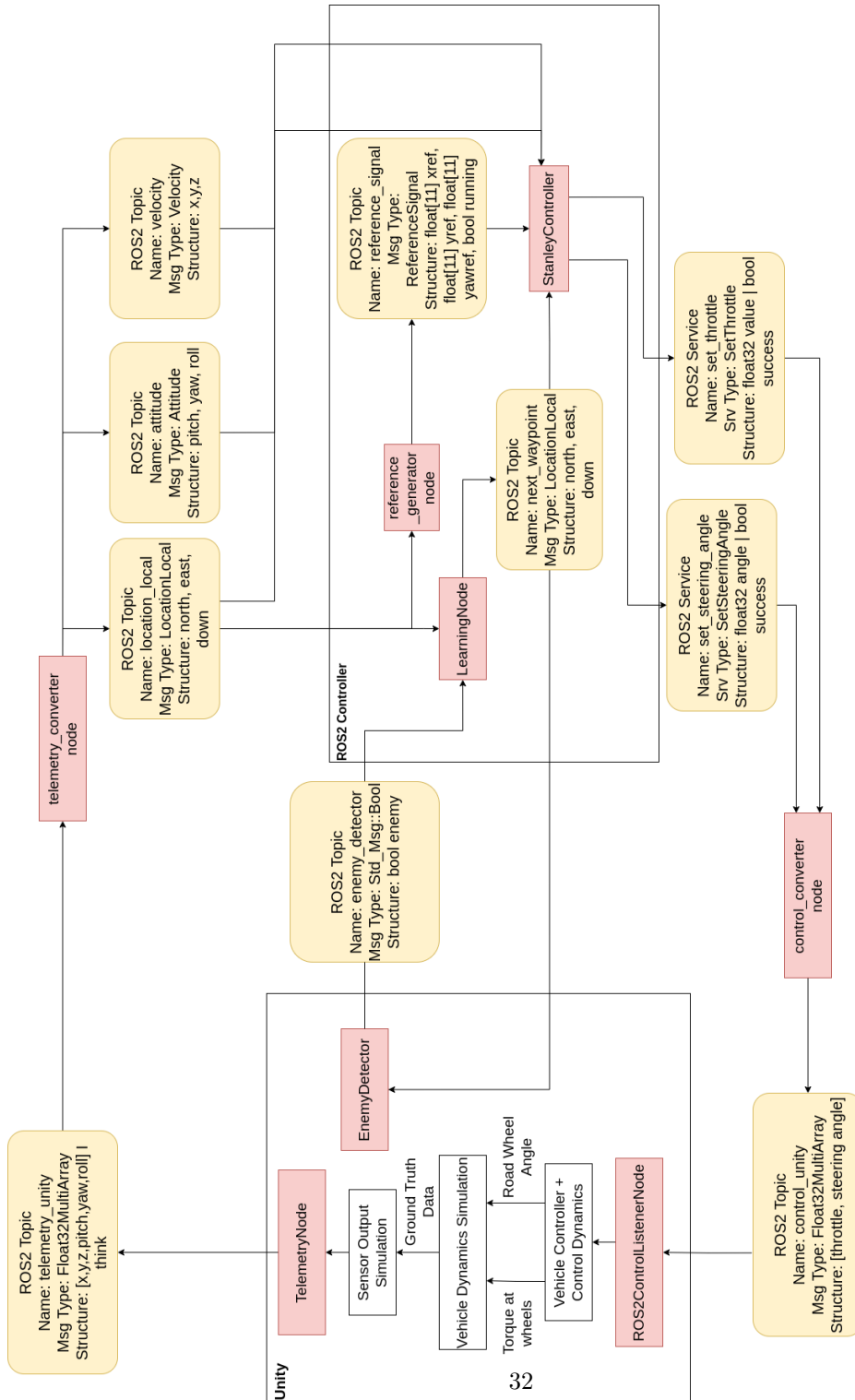
Looking back at the entire research internship another point of improvement comes to mind. During the literature research I would often find useful information, I however sometimes neglected to write down the specific parts of, for example, a paper I found useful. This resulted in me needing to reread some parts of a paper to find the useful part. If I had written down the useful/important parts of a paper while reading it, I could have saved quite some time.

11.3.3 Overall thoughts

Looking ahead at my master’s thesis I have to say that I liked the topic of the internship. The research was relatively practical, I liked seeing intermediate results during the internship. While I would like to do some more academic research during my master’s thesis, I would also really like some practical aspects during the master thesis. Having some practical and semi-tangible results during the thesis will most likely motivate me more than only having results on paper. As I liked the topic I, in consultation with Jozef, decided to have online/runtime monitoring as the main topic of my master thesis. The exact research question to be answered has not yet been specified. One of the contenders is using ComMA as a monitor for AI-related programs.

Appendices

A ROS Nodes architecture of the case study



B TShark arguments used to capture ROS packets

A brief explanation of the arguments supplied to TSHark is listed below:

- **-q** is used to disable the count of captured packages which is printed to the STDOUT by default.
- **-i lo** is used to capture all data from the interface lo(Loopback). If a distributed system would be used, it could be necessary to capture other interfaces as well.
- **-l** is used to flush the stdout(standard output) every time a packet is printed. If this option is not enabled output is only printed if the buffer fills up, resulting in incomplete messages being printed.
- **-Y "rtps.traffic_nature == 3 and rtps.sm.id == 0x15"** is used to filter out none needed RTPS packets. Where `rtps.traffic_nature == 3` denotes that the packet sent is of the type `UNICAST_USERTRAFFIC` and `rtps.sm.id == 0x15` denotes that the packet contains `RTPS_DATA` [30].
- **rtps.param.topicName contains "rq/"** is used to capture all requests.
- **rtps.param.topicName contains "rt/"** is used to capture all message on topics.
- **rtps.param.topicName contains "rr/"** is used to capture all replies.
- **-o rtps.enable_user_data_dissection:TRUE** is used to enable dissection support from TShark, TShark will for a large part deserialize the RTPS message for us which allows us to use the filter used above.
- **-T json** is used to set the output type to JSON, we can select which fields we want to have in the JSON output by specifying the **-e** argument. Currently the following fields are captured:
 - **-e rtps.appId** is used to capture the source and destination of a message.
 - **-e rtps.param.topicName** is used to capture the name of the ROS Topic, Service or Action that the message was sent on.
 - **-e rtps.param.typeName** is used to capture the type of the message sent.
 - **-e rtps.info_ts.timestamp** is used make the timestamp part of the output.
 - **-e rtps.param.serialize.encap_kind** is used make the `encap_kind` part of the output.
 - **-e rtps.param.serialize.encap_len** is used make the `encap_len` part of the output.
 - **-e rtps.issueData** is used make the `issueData` part of the output, the `issueData` contains the actual data sent between ROS nodes.
- **-E separator=, -E quote=d** is used to separate the packets with a comma, resulting in a CSV-file.

C Output of a sniffed packet by TShark

```
1  {
2    "_index": "packets-2022-04-13",
3    "_type": "doc",
4    "_score": null,
5    "_source": {
6      "layers": {
7        "rtps.param.topicName": [
8          "rt/location_local",
9          "rt/location_local"
10       ],
11       "rtps.param.typeName": [
12         "traxxas_stampede_ardupilot_driver_interfaces::msg::dds::
13           LocationLocal_",
14         "traxxas_stampede_ardupilot_driver_interfaces::msg::dds::
15           LocationLocal_"
16       ],
17       "rtps.info_ts.timestamp": [
18         "Apr 13, 2022 21:12:38.461304664 UTC"
19       ],
20       "rtps.param.serialize.encap_kind": [
21         "0x00000001"
22       ],
23       "rtps.param.serialize.encap_len": [
24         "0x00000000"
25       ],
26       "rtps.issueData": [
27         "000000005000000040000006f6e65000400000074776f493020482903fe94"
28       ]
29     }
30   }
```

D TShark arguments used to capture ROS packets

In the table below the conversion of ROS' primitive datatypes to ComMA's datatypes can be found:

ROS Datatype	Comma Datatype
bool	bool
byte	bulkdata< 1 >
char	bulkdata< 1 >
float32	real
float64	real
int8	int8
uint8	uint8
int16	int16
uint16	uint16
int32	int32
uint32	uint32
int64	int64
uint64	Not possible
string	string
wstring	string

E Configuration file for ROS2ComMA

```
1 {
2   "interfaces_to_monitor": [
3     {
4       "name": "ITraxxas",
5       "topics_to_monitor": [
6         {
7           "topic": "location_local",
8           "source": "Simulation",
9           "destination": "Traxxas"
10        },
11        {
12          "topic": "enemy_detector",
13          "source": "EnemyDetector",
14          "destination": "Traxxas"
15        },
16        {
17          "topic": "next_waypoint",
18          "source": "LearningNode",
19          "destination": "Traxxas"
20        }
21      ],
22      "services_to_monitor": [
23      ],
24      "actions_to_monitor": []
25    }
26  ]
27 }
```

F Records before

Monitoring error of des_src_IStateMachine

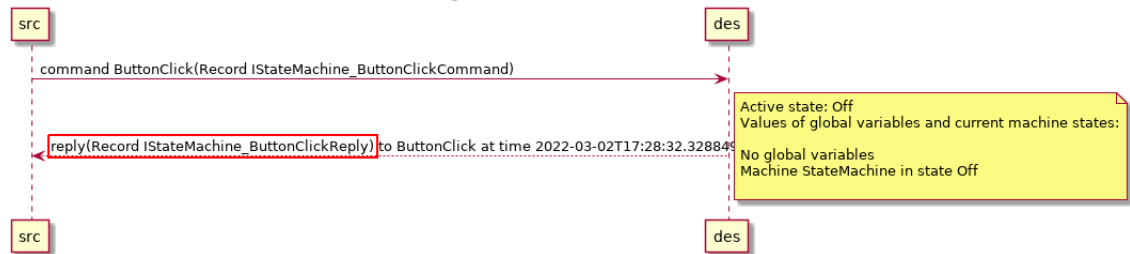
Error description

Interface monitoring error. The event does not conform to the model. Possible events in the current state of the model: Reply to command ButtonClick. Parameters: Record IStateMachine_ButtonClickReply

Error context

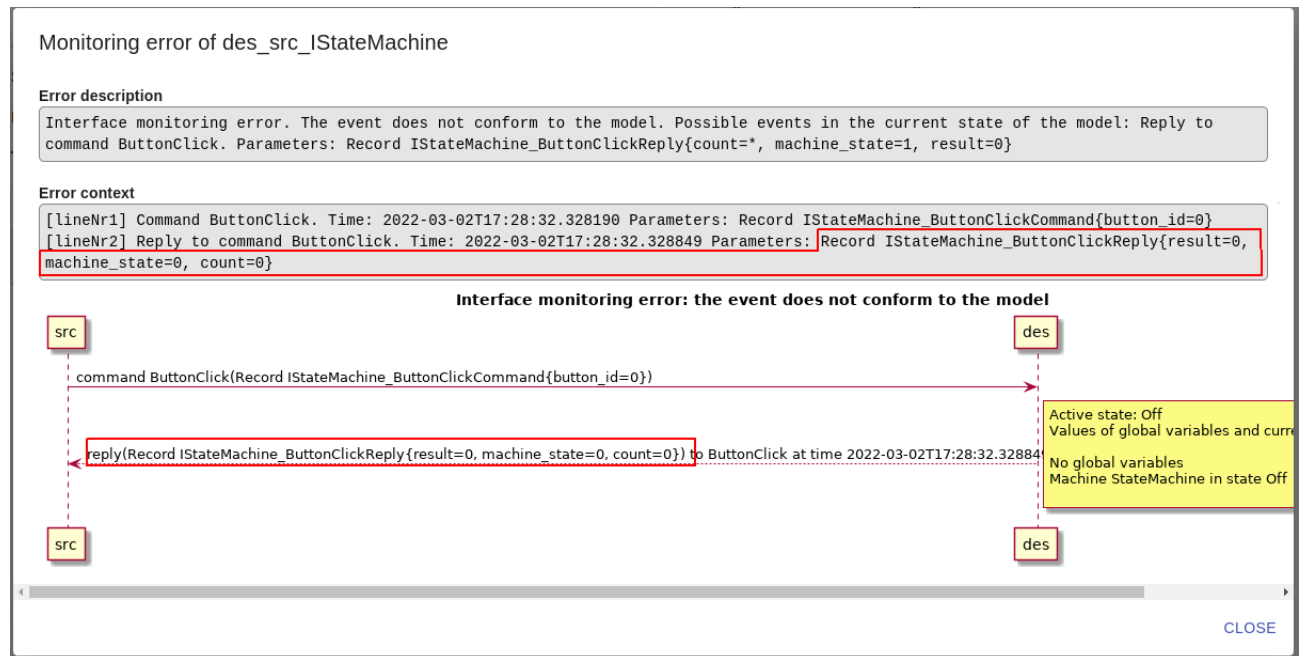
[lineNr1] Command ButtonClick. Time: 2022-03-02T17:28:32.328190 Parameters: Record IStateMachine_ButtonClickCommand
[lineNr2] Reply to command ButtonClick. Time: 2022-03-02T17:28:32.328849 Parameters: Record IStateMachine_ButtonClickReply

Interface monitoring error: the event does not conform to the model



[CLOSE](#)

G Records after



References

- [1] *Robot operating system*. URL: <https://www.ros.org/>.
- [2] *Eclipse CommaSuite*. URL: <https://www.eclipse.org/comma/index.html>.
- [3] Ivan Kurtev, Jozef Hooman, and Mathijs Schuts. “Runtime Monitoring Based on Interface Specifications”. In: Sept. 2017, pp. 335–356. ISBN: 978-3-319-68269-3. DOI: 10.1007/978-3-319-68270-9_17.
- [4] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. eprint: <https://www.science.org/doi/pdf/10.1126/scirobotics.abm6074>. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [5] *Understanding ROS2 Topics*. URL: <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>.
- [6] *Understanding ROS2 Services*. URL: <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>.
- [7] *Understanding ROS2 Actions*. URL: <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>.
- [8] Alexis van Wissen. “Project diagram”. In: 2022.
- [9] Angelo Ferrando et al. “ROSMonitoring: A Runtime Verification Framework for ROS”. In: *Towards Autonomous Robotic Systems*. Ed. by Abdelkhalick Mohammad, Xin Dong, and Matteo Russo. Cham: Springer International Publishing, 2020, pp. 387–399. ISBN: 978-3-030-63486-5.
- [10] *Runtime Monitoring Language*. URL: <https://rmlatdibris.github.io/>.
- [11] Angelo Ferrando et al. “ROSMonitoring: A Runtime Verification Framework for ROS”. In: *Towards Autonomous Robotic Systems*. Ed. by Abdelkhalick Mohammad, Xin Dong, and Matteo Russo. Cham: Springer International Publishing, 2020, pp. 387–399. ISBN: 978-3-030-63486-5.
- [12] Jeff Huang et al. “ROSRV: Runtime Verification for Robots”. In: *Runtime Verification*. Ed. by Borzoo Bonakdarpour and Scott A. Smolka. Cham: Springer International Publishing, 2014, pp. 247–254. ISBN: 978-3-319-11164-3.
- [13] A.Q. Gates et al. “Instrumentation of intermediate code for runtime verification”. In: *28th Annual NASA Goddard Software Engineering Workshop, 2003. Proceedings*. 2003, pp. 66–71. DOI: 10.1109/SEW.2003.1270727.
- [14] *DDS*. URL: <https://www.dds-foundation.org/what-is-dds-3/>.
- [15] *RTPS*. URL: <https://www.eprosima.com/index.php/resources-all/whitepapers/rtps>.
- [16] *ZeroMQ*. URL: <https://zeromq.org/>.
- [17] *Protobuf*. URL: <https://developers.google.com/protocol-buffers>.
- [18] *eProsima Fast DDS*. URL: <https://www.eprosima.com/>.
- [19] *tcpdump*. URL: <https://www.tcpdump.org/>.

- [20] *Wireshark*. URL: <https://www.wireshark.org/>.
- [21] *TShark*. URL: <https://tshark.dev/>.
- [22] *TShark Manual*. URL: <https://www.wireshark.org/docs/man-pages/tshark.html>.
- [23] *ROS-Interfaces*. URL: <https://docs.ros.org/en/rolling/Concepts/About-ROS-Interfaces.html>.
- [24] *rosidl_runtime_py*. URL: https://github.com/ros2/rosidl_runtime_py.
- [25] *rclpy*. URL: <https://github.com/ros2/rclpy>.
- [26] *Synchronous vs. asynchronous service clients*. URL: <https://docs.ros.org/en/foxy/How-To-Guides/Sync-Vs-Async.html>.
- [27] *ComMA JSON-traces*. URL: <https://www.eclipse.org/comma/eventfiles/JSONEventFiles.html>.
- [28] *ComMA tutorial*. URL: <https://www.eclipse.org/comma/tutorial/intro.html>.
- [29] *ComMA traces*. URL: <https://www.eclipse.org/comma/eventfiles/eventfiles.html>.
- [30] *The Real-time Publish-Subscribe Protocol (RTPS) DDS Interoperability Wire Protocol Specification*. URL: <https://www.omg.org/spec/DDS-RTPS/2.2/PDF>.